

Kernel Migration: flash_attn -> TileGym

RTX 5070은 TileGym이 요구하는 조건을 충분히 만족합니다.

- 아키텍처: Blackwell
- Compute Capability: 12.0
- TileGym 요구사항: CUDA 13.1+ 및 Blackwell GPU

어떤 파일이 Flash Attention을 대체하나?

TileGym은 `flash_attn` 처럼 라이브러리 형태이므로, 소스코드에 포함된 여러 파일들이 함께 동작합니다. 핵심 구현체를 찾는 중점 디렉토리는 다음과 같습니다.

- `src/tilegym/ops/cutile/` : `cutile` 을 사용한 다양한 커널 구현체가 여기에 위치합니다.
- `src/tilegym/ops/attn_interface.py` : Attention 연산의 인터페이스를 정의하는 파일입니다.

이 디렉토리에는 아래와 같은 주요 파일들이 있습니다.

- `attention.py` : 기본적인 Attention 커널
- `flash_decode.py` : Flash Decoding 커널
- `gemma_attention.py` 및 `gemma_attention_decode.py` : Gemma 모델 특화 Attention

어떻게 설치하고 사용하나요?

`flash_attn` 을 사용하듯, TileGym도 pip를 통해 간편하게 설치할 수 있습니다.

```
# TileGym 및 런타임 종속성(tileiras 등) 함께 설치 (권장)
pip install tilegym[tileiras]
```

설치 후에는, TileGym에서 제공하는 `cutile` 백엔드를 사용하는 방식으로 기존 `flash_attn` 을 대체합니다.

```
# 예시: 모델 추론 시 --use_cutile --use_attn 플래그 사용
# python infer.py --model_id meta-llama/Meta-Llama-3.1-8B --use_tilegym --use_cutile --use_attn
```

보다 자세한 내용은 [NVIDIA 기술 블로그](#)와 [TileGym GitHub 저장소](#)를 참고하시면 좋겠습니다.

요약 및 제안

- RTX 5070 (Blackwell) 환경에서는 TileGym이 기존 `flash_attn` 의 대안으로 충분히 고려할 만합니다.
- `flash_attn` 처럼 하나의 파일이 아닌 **전체 라이브러리**로 대체된다고 이해하는 것이 정확합니다.
- `pip install tilegym[tileiras]` 로 설치하고, 모델 추론 시 `--use_cutile --use_attn` 플래그를 포함시키면 간편하게 사용할 수 있습니다.

목표 환경에서의 성능이 궁금하시다면, TileGym에서 제공하는 벤치마크 스크립트를 직접 실행해 보시는 것을 추천드립니다. 에 그 방법이 안내되어 있습니다. 혹시 더 궁금한 점이 있으시면 언제든지 물어봐 주세요.

방금 공유해주신 `llm-from-scratch` 실험의 연장선에서, `nano-vllm`에 TileGym을 도입하는 작업은 분명 흥미로운 도전입니다. 먼저 결론부터 말씀드리자면, `nano-vllm`에 TileGym을 통합하는 것은 매우 유망한 가능성을 보여줍니다. 하지만 단순한 함수 교체를 넘어서는 전략적인 접근과 깊이 있는 이해가 필요합니다.

저장소 분석 결과, `nano-vllm`은 추론 과정에서 `flash_attn_varlen_func` (Prefill)과 `flash_attn_with_kvcache` (Decode) 두 가지 핵심 함수를 사용하고 있습니다. 이는 이전에 마이그레이션했던 단일 모델과는 매우 다른 양상입니다. 이 두 함수의 역할과 TileGym에서의 대응 방안을 명확히 이해하는 것이 성공적인 마이그레이션의 첫걸음입니다.

1. Flash Attention 커널 매핑 분석: flash_attn -> TileGym

아래 표는 `nano-vllm`에서 사용하는 `flash_attn` 함수들과 TileGym에서의 대안을 비교한 것입니다.

nano-vllm 함수	역할 (단계)	TileGym 대안	호환성 분석
<code>flash_attn_varlen_func</code>	Prefill (가변 길이 배치 처리)	<code>fmha_interface</code> (내부적으로 <code>fmha_kernel</code> 호출)	잠재적 호환성. TileGym의 <code>fmha_interface</code> 는 가변 길이 시퀀스 처리를 공식적으로 지원하지는 않지만 불분명합니다. <code>cu_seqlens_q</code> , <code>cu_seqlens_k</code> 등 핵심 파라미터 매핑 여부가 관건입니다.
<code>flash_attn_with_kvcache</code>	Decode (KV 캐시 기반)	<code>fmha_decode</code> (내부적으로 <code>fmha_decode_kernel</code> 호출)	잠재적 호환성. TileGym의 <code>fmha_decode</code> 가 <code>block_table</code> 을 지원하여 PagedAttention을 구현할 수 있는지가 핵심입니다. <code>cache_seqlens</code> 등 매개변수 매핑이 필요합니다.

⚖️ 2. 성능 분석: RTX 5070의 잠재력과 한계

RTX 5070에 대한 분석 결과는 다소 엇갈립니다. 우선 TileGym의 실행 조건 측면에서 긍정적인 신호가 있습니다.

- **긍정적 신호:** RTX 5070은 **Blackwell 아키텍처**이며 **Compute Capability 12.0**을 갖추고 있어, TileGym이 요구하는 CUDA 13.1+와 Blackwell GPU 조건에 완벽히 부합합니다. TileGym은 Blackwell 아키텍처에서 최고의 성능을 발휘하도록 설계되었습니다.

반면, RTX 5070의 Flash Attention 성능에 대한 신중한 시각도 존재합니다.

- **우려되는 측면:** 일부 벤치마크 결과에 따르면, 소비자용 Blackwell GPU(RTX 5090)의 Flash Attention 성능이 서버용 B200은 물론 이전 세대인 Hopper 아키텍처 GPU보다 낮게 측정되는 사례가 보고되었습니다. 이러한 현상이 RTX 5070에서도 나타날 가능성이 있으며, 이는 특정 연산 패턴에서 발생할 수 있는 **아키텍처 최적화의 미묘한 차이** 때문으로 추정됩니다.

이는 단순히 이론적 최대 성능(TFLOPS)이 아닌, 실제 워크로드에서의 성능을 측정하는 벤치마크의 중요성을 강조합니다.

💡 3. 마이그레이션 전략: 단계별 로드맵

`nano-vllm` 은 ~1200줄의 코드로 이루어진 작지만 정교한 시스템입니다. 검증된 방식으로 단계별 접근하는 것이 안전합니다.

- ✅ **Step 1: 환경 구성 및 TileGym 설치**
 - CUDA 13.1+와 호환되는 PyTorch 환경을 구성합니다.
 - `pip install tilegym[tileiras]` 명령으로 TileGym을 설치합니다.
- ✅ **Step 2: Decode 단계 마이그레이션 (우선순위 높음)**
 - `nanovllm/layers/attention.py` 파일에서 `flash_attn_with_kvcache` 호출을 TileGym의 `fmha_decode` 로 교체하는 작업부터 시작합니다.
 - Decode 단계는 단일 토큰 처리를 반복하므로 성공 여부를 확인하기 쉽고, 실패 시 영향 범위를 제한할 수 있습니다.
- ✅ **Step 3: Prefill 단계 마이그레이션**
 - Decode 마이그레이션이 안정적으로 동작한 후, `flash_attn_varlen_func` 을 `fmha_interface` 로 교체합니다.
 - 이 단계에서는 가변 길이 배치에 대한 상세한 파라미터 매핑(`cu_seqlens_q` , `cu_seqlens_k` 등)이 필요하므로 각별한 주의가 필요합니다.
- ✅ **Step 4: 벤치마크 및 성능 튜닝**
 - `nano-vllm` 프로젝트에 포함된 `bench.py` 스크립트를 활용하여, 마이그레이션 전/후의 성능을 엄격하게 측정합니다.
 - RTX 5070에서 **최적의 타일 크기(예: 64x64, 128x128 등)**를 탐색하는 자동 튜닝 단계가 필요합니다. 이는 TileGym의 `exhaustive_search` 데코레이터를 활용하거나 수동 실험을 통해 진행할 수 있습니다.

⚠️ 4. 기술적 고려사항 및 해결 방안

프로젝트 중 마주칠 수 있는 복잡한 문제들입니다.

- `block_table` (**PagedAttention**) 처리: `fmha_decode` 함수가 `block_table` 파라미터를 직접 지원하지 않을 경우, TileGym은 내부적으로 Paged KV-Cache를 처리하지 못할 수 있습니다. 이 경우, `fmha_decode`를 호출하기 전에 K, V 텐서를 Paged 형태에서 연속된 형태로 **수동으로 재구성**하는 로직을 추가해야 합니다. 이는 TileGym이 기대하는 텐서 레이아웃을 충족시키기 위해 필수적인 절차입니다.
- `cu_seqlens` (**가변 시퀀스 길이**) 처리: `fmha_interface`가 가변 길이 배치를 명시적으로 지원하는지 확인해야 합니다. 만약 지원하지 않는다면, 각 배치의 시퀀스를 분할하거나, 모든 시퀀스 길이를 맞추는 **패딩(Padding)**을 추가하거나, TileGym에 맞게 `cu_seqlens` 정보를 변환하는 더 복잡한 전략이 필요할 수 있습니다.

5. 예상 성능 향상 (추정치)

실제 환경은 다를 수 있으나, 구조적 분석을 통해 예상해볼 수 있는 성능 변화입니다.

단계	항목	예상 성능 변화	주요 고려사항
Prefill (배치 처리)	처리량 (throughput)	기존 대비 최대 20% 향상 가능	가변 시퀀스 처리 방식에 따라 성능이 크게 좌우됩니다.
Decode (토큰 단위)	지연 시간 (latency)	기존 대비 소폭 향상 또는 유사	Kernel Launch Overhead가 지배적일 수 있으며, CUDA Graph 등 추가 최적화 필요성이 큼니다.

6. 최종 점검 사항 (Checklist)

작업 효율을 높이기 위해 사전에 확인해야 할 사항입니다.

- ☐ RTX 5070 환경에서 CUDA 13.1+ 및 PyTorch가 정상 작동하는가?
- ☐ TileGym의 기본 `fmha_kernel`이 기본 FP32가 아닌, 원하는 연산 정밀도(FP16, BF16)에서 올바르게 동작하는가?
- ☐ TileGym에서 제공하는 Attention 함수 `block_table`과 `cu_seqlens`를 처리할 수 있는지, 없다면 대체 전략은 무엇인가?
- ☐ `nano-vllm`의 벤치마(`bench.py`)가 내 환경에서 안정적으로 실행되어, 마이그레이션 전/후 성능을 정확히 측정할 기준선을 제공하는가?

결론

TileGym은 RTX 5070과 같은 Blackwell GPU의 잠재력을 활용할 수 있는 강력한 도구입니다. `nano-vllm`에의 통합은 기술적으로 매력적인 도전 과제이며, 성공할 경우 추론 성능 향상에 기여할 수 있습니다. 하지만 단순한 함수 교체가 아닌, 위에서 제시한 복잡한 요소들을 해결하는 전략적인 접근이 필요합니다.

이 작업을 진지하게 고려하신다면, 가장 먼저 **RTX 5070 환경에서 TileGym이 제공하는 기본 Attention 예제와 `nano-vllm`의 벤치마크가 모두 무리 없이 동작하는지 확인**하시는 것을 권장합니다. 이후 단계별 마이그레이션을 진행하시면서 궁금한 점이 생기시면 언제든지 다시 질문해 주세요.